Monthly Release Plan: EZ Metric

Deliver centralized admin configuration panel, perform full system QA, fix all bugs, and deploy to production.

Duration: 10 weeks (3 monthly releases) | Prepared by: Saul Goodman | Date: 2026-03-16

Team

#	Role	Name	Monthly Focus
1	Frontend	TBD	UI/UX for all modules
2	Backend	TBD	APIs, business logic, integrations

***Note:** No dedicated QA — both developers handle testing on Day 5 of each week.*

Monthly Roadmap Overview

Month	Weeks	Focus	Key Deliverables
Month 1	1–4	Core Platform	Auth, Dashboard, Clock System, Shift Management, Face ID
Month 2	5–8	Business Logic & Reporting	Efficiency, Bonus, Salary, Reporting, Telegram Bot
Month 3	9–10	Admin Panel, QA & Launch	Admin Config, Final QA, Bug Fixes, Deployment

Month 1: Core Platform

Week 1 — Authentication & Role Management

Sprint Goal: Fully functional admin and manager login with role-based access control

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
1	Build admin login page with form validation	1	Login UI with email/password fields, error states, loading indicator	—

2	Build manager login page with role selection	1–2	Login page supporting admin/manager role switch	—
3	Implement auth state management & session handling	2–3	Token storage, auto-logout on expiry, protected route wrapper	#5 (Auth API)
4	Build app shell — sidebar navigation, header, role-based menu items	3–4	Responsive layout shell with nav links scoped by role (admin vs manager)	#6 (Role API)

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
5	Build auth API — login endpoint with JWT token generation	1	POST /api/auth/login — validates credentials, returns JWT + refresh token	—
6	Build role & permission system	1–2	Role model (admin/manager), permission middleware, route guards	—
7	Build user management API — CRUD for admin/manager accounts	2–3	/api/users — create, list, update, deactivate accounts with role assignment	—
8	Set up database schema, project structure, CI/CD pipeline	3–4	DB migrations for users/roles, project scaffolding, automated deploy pipeline	—

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test admin login flow (valid/invalid	Frontend	Tasks #1, #3

	credentials, session expiry)		
2	Test manager login flow and role scoping	Frontend	Tasks #2, #4
3	Test auth API edge cases (wrong password, expired token, refresh flow)	Backend	Tasks #5, #6
4	Test user CRUD operations	Backend	Task #7
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 2 — Dashboard & Clock System (Part 1)

Sprint Goal: Live dashboard with worker status and core clock-in/clock-out functionality

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
9	Build main dashboard page with summary widgets	1–2	Dashboard layout with total hours, attendance count, efficiency summary cards	—
10	Build real-time worker status cards (on-shift / off-shift)	2–3	Worker cards showing live status with auto-refresh polling	#14 (Worker status API)
11	Build clock-in / clock-out UI for workers	3–4	Clock button with confirmation, timestamp display, current status indicator	#13 (Clock API)
12	Build worker profile form (name, photo, salary type)	4	Worker registration form with photo upload and salary type dropdown	#15 (Worker API)

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
13	Build clock-in/clock-out API with timestamp storage	1–2	POST /api/clock/in, POST /api/clock/out — records timestamps, validates sequence	—
14	Build worker status API (on-shift / off-shift real-time)	2–3	GET /api/workers/status — returns current status for all workers	#13
15	Build worker profile CRUD API	3	/api/workers — create/update worker with name, photo URL, salary type	—
16	Build dashboard aggregation API (summary stats)	3–4	GET /api/dashboard/summary — total hours, attendance, efficiency for today/week	#13

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test clock-in/out full flow end-to-end	Both	Tasks #11, #13
2	Test dashboard data accuracy and auto-refresh	Frontend	Tasks #9, #10, #16
3	Test worker profile creation with photo upload	Both	Tasks #12, #15
4	Test worker status API under concurrent clocking	Backend	Task #14
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 3 — Clock System (Part 2) & Shift Management (Part 1)

Sprint Goal: Complete clock system with validations, grace period, manual edits; start shift definitions

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
17	Build invalid clock action error handling UI (double clock-in, wrong sequence)	1	Error toasts and blocked states for invalid clock actions	#21 (Validation API)
18	Build manual clock edit form with notes field	1–2	Admin form to override clock-in/out time with required reason note	#22 (Manual edit API)
19	Build clock-in note input and auto/manual status indicator	2–3	Note field on clock action, badge showing "Auto" vs "Manual" per record	—
20	Build shift definition CRUD page (create, edit, delete shifts)	3–4	Shift management page with form for start time, end time, break duration	#24 (Shift API)

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
21	Add clock validation logic (double clock-in, incorrect sequence prevention)	1	Validation middleware on clock endpoints returning descriptive errors	—
22	Build manual clock edit API with audit trail	1–2	PUT /api/clock/:id — admin edits clock record,	—

			stores original value + edit note	
23	Implement grace period rules engine	2–3	Grace period config model, auto-apply on clock-in evaluation	—
24	Build shift CRUD API (create, edit, delete shift definitions)	3–4	/api/shifts — full CRUD with start time, end time, break duration fields	—

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test invalid clock action handling (try double clock-in, wrong sequence)	Both	Tasks #17, #21
2	Test manual clock edit with audit trail	Both	Tasks #18, #22
3	Test grace period rule application	Backend	Task #23
4	Test shift CRUD operations	Both	Tasks #20, #24
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 4 (Month 1 QA) — Shift Calculations, Face ID & Month 1 Stabilization

Sprint Goal: Complete shift hour calculations, deliver Face ID placeholder, stabilize all Month 1 features

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
25	Build daily worked hours display per worker	1	Worker detail view showing calculated hours, overtime, breaks deducted	#29 (Hours calc API)

26	Build Face ID verification UI with camera placeholder	1–2	Face ID screen with camera capture area, verification status badge	—
27	Build clock records history page with Excel export button	2–3	Paginated clock history table per worker with "Export to Excel" action	#31 (Export API)
28	Month 1 bug fixing — fix all UI issues from Weeks 1–3 testing	3–4	All critical/major frontend bugs resolved	—

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
29	Build daily worked hours calculation engine (shift-based, break deduction, overtime)	1–2	Service calculating net hours = clock time − breaks, flags overtime beyond shift	—
30	Build Face ID API integration architecture (API-ready hooks)	2	POST /api/faceid/verify — placeholder endpoint, status storage, ready for real API	—
31	Build clock records export to Excel	2–3	GET /api/clock/export?format=xlsx — generates Excel file with clock history	—
32	Month 1 bug fixing — fix all API/logic issues from Weeks 1–3 testing	3–4	All critical/major backend bugs resolved	—

🧪 Day 5 — Month 1 Integration Testing

#	Activity	Owner	Scope

1	End-to-end test: login → dashboard → clock-in → view hours	Both	Full Month 1 flow
2	Test shift hour calculations with various shift configs	Backend	Task #29
3	Test Face ID UI states (verified / not verified / pending)	Frontend	Task #26
4	Test Excel export with real clock data	Both	Tasks #27, #31
5	Regression test all auth and clock features	Both	Weeks 1–3 features
6	Code review, merge all PRs, tag Month 1 release	Both	All PRs

Month 1 Deliverables Checklist

- ☐ Admin & manager login with role-based access
- ☐ Main dashboard with real-time worker status
- ☐ Clock-in / clock-out with validations, grace period, manual edits
- ☐ Shift CRUD with worked hours calculation, overtime, break deduction
- ☐ Face ID verification UI & API-ready architecture
- ☐ Clock records history with Excel export

Month 2: Business Logic & Reporting

Period: TBD (Weeks 5–8)

Release Goal: Deliver efficiency tracking, bonus calculation, full salary management, reporting suite, and Telegram bot — making EZ Metric a complete payroll management system.

Week 5 — Efficiency Calculation & Bonus Rules (Part 1)

Sprint Goal: Working efficiency calculator with visualizations and bonus rule engine

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
33	Build worker efficiency table	1–2	Table showing per-worker	#37 (Efficiency API)

	(worked hours, expected hours, percentage)		efficiency with calculated percentage column	
34	Build efficiency progress bar visualization	2	Visual progress bar component per worker, color-coded by threshold	—
35	Build bonus rules management page (create formula-based and fixed rules)	3–4	Bonus rule CRUD page with formula editor and fixed amount option	#39 (Bonus API)
36	Build bonus progress visualization and amount display with currency	4	Progress bar toward next bonus tier, formatted currency display	—

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
37	Build efficiency calculation engine (worked vs expected hours, percentage)	1–2	GET /api/efficiency — calculates efficiency per worker for date range	—
38	Build efficiency history API for trend tracking	2	GET /api/efficiency/history — historical efficiency data for charts	—
39	Build bonus rule CRUD API (formula-based and fixed)	2–3	/api/bonus-rules — create/edit/delete rules with formula or flat amount	—
40	Build bonus matching engine (hours vs rules → calculated bonus)	3–4	Service matching worked hours to bonus tiers, returns calculated amounts	#39

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test efficiency calculation accuracy with various hour combinations	Both	Tasks #33, #37
2	Test efficiency progress bar rendering at edge values (0%, 50%, 100%, >100%)	Frontend	Task #34
3	Test bonus rule creation (formula + fixed) and matching logic	Both	Tasks #35, #39, #40
4	Test bonus display with different currencies	Frontend	Task #36
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 6 — Bonus Completion & Salary Calculation (Part 1)

Sprint Goal: Finalize bonus system, build salary rate configuration and overtime pay calculation

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
41	Build bonus amount display per worker with currency formatting	1	Worker salary view showing matched bonus with localized currency	#40
42	Build salary rate configuration UI (hourly, percentage, flat)	1–2	Admin form to set rate type and value per worker	#46 (Salary rate API)
43	Build overtime pay display	2–3	Overtime hours and additional pay shown in	#47 (OT pay API)

	integrated with salary view		worker salary breakdown	
44	Build loan & pre-payment input form	3–4	Form to record advances/loans per worker with amount and date	#48 (Loan API)

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
45	Finalize bonus calculation integration with salary pipeline	1	Bonus amounts feed into salary calculation service	—
46	Build salary rate configuration API (hourly, percentage, flat rate)	1–2	/api/salary/rates — configure rate type and value per worker	—
47	Build overtime pay calculation service	2–3	Service calculating additional pay from overtime hours × rate multiplier	—
48	Build loan & pre-payment tracking API	3–4	/api/loans — record advances, track balance, integrate with salary deductions	—

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test bonus calculation end-to-end (rule → match → amount display)	Both	Tasks #41, #45
2	Test all three salary rate types (hourly, percentage, flat)	Both	Tasks #42, #46
3	Test overtime pay calculation with edge cases	Backend	Task #47

4	Test loan recording and balance tracking	Both	Tasks #44, #48
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 7 — Salary Calculation (Part 2) & Reporting (Part 1)

Sprint Goal: Complete salary deductions, late penalties, manual edits; start reporting module

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
49	Build salary deductions & distribution rules UI	1	Admin config page for deduction categories and distribution rules	#53 (Deductions API)
50	Build late charge-back display (per-minute penalty view)	1–2	Worker salary view showing late minutes and deducted amount	#54 (Late penalty API)
51	Build manual salary edit form with override capability	2–3	Admin form to adjust final salary with reason field	#55 (Salary edit API)
52	Build overall salary totals report page	3–4	Summary report page showing total payroll across all workers	#56 (Report API)

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
53	Build salary deductions & distribution rules engine	1–2	Configurable deduction categories, percentage/fixed splits	—
54	Build late charge-back calculation (per-minute penalty)	2	Service calculating late minutes × penalty rate,	—

			auto-deducts from salary	
55	Build manual salary override API with audit trail	2–3	PUT /api/salary/:id/override — admin adjusts salary, stores reason + original	—
56	Build overall salary totals report API	3–4	GET /api/reports/salary-totals — aggregated payroll data with filters	—

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test salary deduction scenarios (multiple deduction types combined)	Both	Tasks #49, #53
2	Test late penalty calculation with various late durations	Both	Tasks #50, #54
3	Test manual salary override and audit trail	Both	Tasks #51, #55
4	Test salary totals report data accuracy	Both	Tasks #52, #56
5	Fix bugs found during testing	Both	All issues
6	Code review & merge all PRs	Both	All PRs

Week 8 (Month 2 QA) — Reporting Completion, Telegram Bot & Month 2 Stabilization

Sprint Goal: Complete reporting suite, deliver Telegram bot, stabilize all Month 2 features

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
57	Build filter-based report analysis page (date,	1	Report page with filter bar — date range picker, worker	#63 (Filter API)

	worker, department)		dropdown, department	
58	Build detailed per-worker report page	1–2	Individual worker report showing hours, efficiency, salary, bonuses	#64 (Worker report API)
59	Build daily/monthly/yearly trend charts (hours & efficiency history)	2–3	Chart components showing worked hours and efficiency trends over time	#65 (Trends API)
60	Build Excel export UI for salary & clock data	3	Export buttons on report pages with format selection and download	#66 (Export API)
61	Month 2 bug fixing — fix all UI issues from Weeks 5–7	3–4	All critical/major frontend bugs resolved	—

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
62	Build Telegram bot setup & worker account linking	1	Bot registration, /start command, link worker ID to Telegram chat	—
63	Build report filter API (date range, worker, department)	1–2	GET /api/reports with query params for all filter combinations	—
64	Build per-worker detailed report API	2	GET /api/reports/worker/:id — hours, efficiency, salary, bonus breakdown	—
65	Build trend data API (daily/monthly/yearly aggregations)	2–3	GET /api/reports/trends — time-series data for charts	—

66	Build salary & clock data Excel export API	3	GET /api/reports/export?format=xlsx — generates comprehensive Excel report	—
67	Build Telegram vacation request flow & payment statement delivery	3–4	/vacation command with admin notification, /statement command with PDF	—
68	Build weekly statement auto-generation & admin email	4	Scheduled job generating weekly summary, sends to admin email	—

🧪 Day 5 — Month 2 Integration Testing

#	Activity	Owner	Scope
1	End-to-end test: efficiency → bonus → salary → report flow	Both	Full Month 2 pipeline
2	Test all report filters and per-worker report accuracy	Both	Tasks #57, #58, #63, #64
3	Test trend charts with various date ranges	Frontend	Tasks #59, #65
4	Test Excel export data integrity	Both	Tasks #60, #66
5	Test Telegram bot — vacation request + payment statement	Backend	Tasks #62, #67
6	Test weekly statement email delivery	Backend	Task #68
7	Regression test efficiency, bonus, salary modules	Both	Weeks 5–7 features
8	Code review, merge all PRs, tag Month 2 release	Both	All PRs

Month 2 Deliverables Checklist

☐ Efficiency calculator with progress visualization

☐ Bonus rules engine with formula & fixed rules, progress display
☐ Salary calculation — all rate types, overtime, loans, deductions, late penalties
☐ Manual salary override with audit trail
☐ Reporting suite — totals, per-worker, filters, trends, Excel export
☐ Telegram bot — vacation requests, payment statements
☐ Weekly statement auto-generation & admin email

Month 3: Admin Panel, Final QA & Launch

Period: TBD (Weeks 9–10)

Release Goal: Deliver centralized admin configuration panel, perform full system QA, fix all bugs, and deploy to production.

Week 9 — Admin Configuration Panel

Sprint Goal: Centralized admin panel to manage all system settings (shifts, salary, bonus, global config)

🖥️ Frontend — TBD

#	Task	Day	Deliverable	Depends On
69	Build admin shift configuration management page	1	Centralized page to view/edit/delete all shift definitions	—
70	Build admin salary rules management page	1–2	Page to manage rate types, deduction rules, penalty config per worker	—
71	Build admin bonus rules management page	2–3	Page to manage all bonus formulas and fixed rules in one place	—
72	Build system settings page	3–4	Global settings form with	#76 (Settings API)

	(grace period, currency, language)		save/reset, affects entire system	

⚙️ Backend — TBD

#	Task	Day	Deliverable	Depends On
73	Build admin shift config API (bulk operations, validation)	1	/api/admin/shifts — list all, bulk update, validate no conflicts	—
74	Build admin salary rules API (centralized management)	1–2	/api/admin/salary-rules — manage all rate configs and deduction rules	—
75	Build admin bonus rules API (centralized management)	2–3	/api/admin/bonus-rules — manage formulas, test rule matching	—
76	Build system settings API (grace period, currency, language, preferences)	3–4	/api/admin/settings — global config CRUD, applies system-wide	—

🧪 Day 5 — Testing & Stabilization

#	Activity	Owner	Scope
1	Test admin shift management (create, edit, delete, conflict detection)	Both	Tasks #69, #73
2	Test admin salary rules management	Both	Tasks #70, #74
3	Test admin bonus rules management and rule testing feature	Both	Tasks #71, #75
4	Test system settings and verify changes propagate system-wide	Both	Tasks #72, #76
5	Fix bugs found during testing	Both	All issues

6	Code review & merge all PRs	Both	All PRs

Week 10 — Full QA, Bug Fixing & Production Deployment

Sprint Goal: Stabilize the entire system, fix all remaining bugs, deploy to production, hand over to client

Days 1–2: End-to-End Integration Testing

#	Test Area	Owner	Scope
1	Full user journey: login → clock-in → shift → hours → efficiency → bonus → salary → report	Both	All modules connected
2	Admin journey: configure shifts → set salary rules → set bonus rules → view reports → export	Both	Admin panel + reports
3	Telegram bot: vacation request → admin notification → approval flow	Backend	Telegram integration
4	Excel exports: clock records, salary reports, filtered reports	Both	All export endpoints
5	Face ID: verification flow, status display, fallback handling	Frontend	Face ID module
6	Edge cases: midnight clock-in, timezone handling, concurrent users	Backend	System reliability

Days 2–3: Bug Fixing & Stabilization

#	Activity	Owner	Priority
1	Fix critical bugs from integration testing	Both	🔴 Critical
2	Fix major bugs (incorrect calculations, broken flows)	Both	🟡 Major

3	Fix minor UI/UX issues (alignment, responsiveness, copy)	Frontend	🟢 Minor
4	Performance optimization (slow queries, large data sets)	Backend	🟡 Major

Days 3–4: UAT & Release Preparation

#	Activity	Owner	Day
1	Code freeze — no new features	Both	3
2	Final regression pass on all modules	Both	3–4
3	Deploy to staging environment	Backend	4
4	Client UAT walkthrough with EZ Truck Repair	PM	4
5	Address UAT feedback (critical items only)	Both	4

Day 5: Production Deployment & Handover

#	Activity	Owner
1	Production server setup & deployment	Backend
2	DNS configuration & SSL setup	Backend
3	Post-deploy smoke test — all critical flows	Both
4	Telegram bot production activation	Backend
5	Admin account creation for client	Backend
6	Client training walkthrough	PM
7	Handover documentation & source code transfer	PM

Final Release Checklist

All Modules

- ☐ Authentication & role management (admin, manager)
- ☐ Main dashboard with real-time worker status

☐ Clock-in / clock-out with validations, grace period, manual edits, notes

☐ Shift management with hour calculation, overtime, break deduction

☐ Face ID verification UI & API-ready architecture

☐ Efficiency calculator with progress visualization

☐ Bonus rules engine (formula + fixed) with matching and progress display

☐ Salary calculation — hourly/percentage/flat rates, overtime, loans, deductions, late penalties

☐ Manual salary override with audit trail

☐ Reporting — totals, per-worker, filters, trends, Excel export

☐ Telegram bot — vacation requests, payment statements

☐ Weekly statement auto-generation & admin email

☐ Admin configuration panel (shifts, salary, bonus, system settings)

Non-Functional

☐ All critical/major bugs resolved

☐ Excel exports tested with real data

☐ Production deployment complete

☐ Post-deploy smoke test passed

☐ Client UAT sign-off received

☐ Source code and documentation handed over

Dependencies & Risks (Project-Level)

#	Dependency / Risk	Impact	Mitigation
1	No dedicated QA — devs self-test on Day 5	🟡 Medium	Structured test checklists per module, cross-testing (FE tests BE flows and vice versa)
2	Face ID API integration depends on client providing API keys	🟡 Medium	Build API-ready architecture with mocks; integrate real API when keys provided
3	Telegram bot requires bot token and server with public URL	🟢 Low	Set up early in Week 8; use ngrok for dev testing
4	Salary calculation complexity — many	🟡 Medium	Build incrementally (Week 6–7), test each

	rate types and deduction rules		rate type independently
5	Excel export performance with large datasets	🟢 Low	Stream generation, pagination, test with 1000+ records
6	Month 2 has heaviest workload (efficiency + bonus + salary + reporting)	🟡 Medium	Strict scope control, carry over non-critical items to Week 9 if needed

Monthly Summary *(filled at end of each month)*

Metric	Month 1	Month 2	Month 3	Total
Tasks planned	32	36	8 + QA	76+
Tasks completed	—	—	—	—
Bugs found	—	—	—	—
Bugs fixed	—	—	—	—
Carry-over	—	—	—	—

Status	Value
Release status	⏳ Not Started
Features delivered	0 of 13 modules
Known issues remaining	—